Chapter 5 - Functions

How Do Functions Work in Python?

A function is a named, reusable block of code that performs one task. You define it once with the def keyword and call it as many times as you want. When a function is called, Python jumps into the function body, runs it with the arguments you passed, and then returns to the place where it was called, handing back the value given by the return statement. A function without return gives back None.

```
def greet(name):
    """Return a greeting message."""
    return f"Hello, {name}!"

message = greet("Ayesha")
print(message)   # Hello, Ayesha!
```

Parameters and Arguments

Parameters are the names listed in the function definition; arguments are the actual values passed when calling it. Python supports positional arguments, keyword arguments and default values.

```
def power(base, exponent=2):
    return base ** exponent

print(power(5))                 # 25 (default exponent)
print(power(2, 3))              # 8  (positional)
print(power(exponent=3, base=4)) # 64 (keyword)
```

*args and **kwargs

Use *args to accept any number of positional arguments (collected in a tuple) and **kwargs to accept any number of keyword arguments (collected in a dictionary).

```
def total(*numbers):
    return sum(numbers)

print(total(1, 2, 3, 4))   # 10
```

Variable Scope

Variables created inside a function are local and exist only while the function runs. Variables created outside are global. Python looks up names in the order Local, Enclosing, Global, Built-in (the LEGB rule).

Lambda Functions and Recursion

A lambda is a small anonymous function written in one line, such as lambda x: x * 2. Recursion is when a function calls itself; it needs a base case to stop, for example when computing a factorial.

```
square = lambda x: x * x

def factorial(n):
    if n <= 1:          # base case
        return 1
    return n * factorial(n - 1)
```

```
print(factorial(5))   # 120
```

Why Use Functions?

- Reusability: write once, use many times.
- Organisation: break a big problem into small, named pieces.
- Easier testing and debugging.
- Cleaner, shorter and more readable programs.